SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Solutions with Python Programs and Output Screenshots

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Weightage:**	40% of CIA
Student Name:	Aryan.K.S	**Reg. No:**	192325051
Section / Batch:		**Date:**	

Question 1 | Section 1: Regular Expressions – Resume Information Extraction

Question

A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.
- Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Answer *Python program: q1_resume_extraction.py*

```
"""Q1 - Resume Information Extraction System using Regular Expressions."""
import re

SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

# ---------- Regular-expression patterns ----------
NAME_RE = re.compile(r"^Name\s*:\s*([A-Za-z][A-Za-z .]+?)\s*$", re.MULTILINE)
EMAIL_RE = re.compile(r"\b[A-Za-z0-9. %+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
MOBILE_RE = re.compile(r"(?<!\d)(?:\+91[-\s]?)?([6-9]\d{4}[-\s]?\d{5})(?!\d)")
EXP_RE = re.compile(
    r"experience\s*:\s*(\d+(?:\.\d+)?)\s*(?:years?|yrs?)"            # Experience: 3 years
    r"|(\d+(?:\.\d+)?)\+?\s*(?:years?|yrs?)\s+of\s+experience",       # 3 years of experience
    re.IGNORECASE)

RESUMES = [
    """Name: Priya Sharma
Email: priya.sharma@gmail.com | Mobile: +91-9876543210
Skills: Python, SQL, Machine Learning, NLP
Experience: 5 years as a Data Scientist""",
    """Name: Rahul Verma
Contact: rahul v@outlook.com, 98765 43211
Skills: Java, SQL, JavaScript, Spring
Experience: 4 years""",
    """Name: Ananya Iyer
Email: ananya.iyer@simats.edu | Phone: 9123456780
Technical skills: python, nlp, machine learning
Summary: Engineer with 2 years of experience in text mining.""",
    """Name: Karthik Raj
Email: karthik.raj@yahoo.in | Mobile: 8012345678
Skills: Python, Java
Experience: 1.5 years""",
]
```

```python
# ---------- Extraction functions ----------
def extract name(text):
    m = NAME RE.search(text)
    return m.group(1) if m else "Not found"

def extract emails(text):
    return EMAIL RE.findall(text)

def extract mobiles(text):
    return [re.sub(r"\D", "", m) for m in MOBILE RE.findall(text)]

def extract skills(text):
    # \b ensures "Java" does not match inside "JavaScript"
    return [s for s in SKILLS if re.search(r"\b" + re.escape(s) + r"\b", text, re.IGNORECASE)]

def extract experience(text):
    m = EXP RE.search(text)
    return float(m.group(1) or m.group(2)) if m else 0.0

def build profile(text):
    return {"name": extract name(text), "emails": extract emails(text),
            "mobiles": extract mobiles(text), "skills": extract skills(text),
            "experience": extract experience(text)}

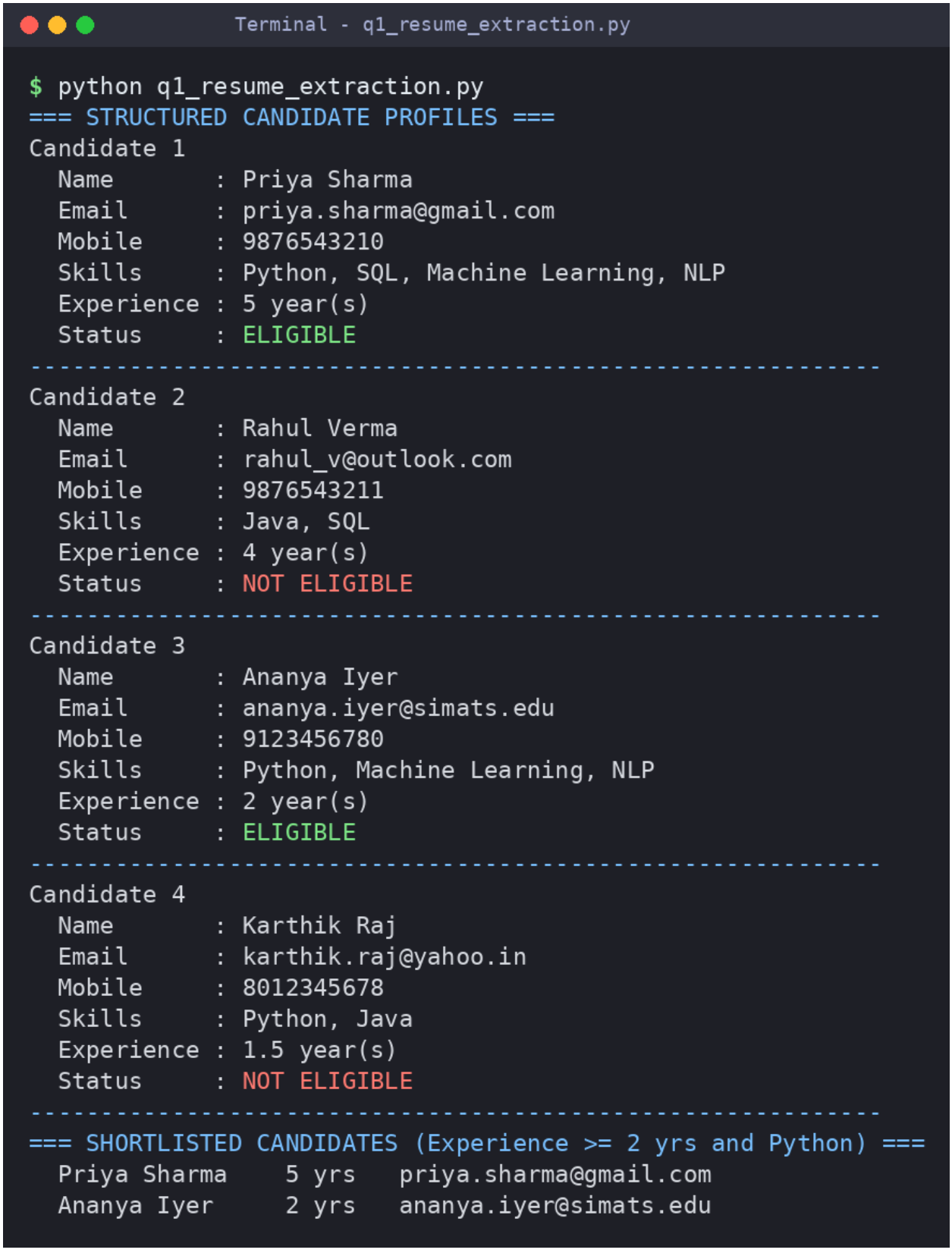
def is eligible(p):
    """Minimum eligibility: at least 2 years of experience AND Python skill."""
    return p["experience"] >= 2 and "Python" in p["skills"]

def show profile(no, p):
    print(f"Candidate {no}")
    print(f"  Name       : {p['name']}")
    print(f"  Email      : {', '.join(p['emails']) or 'Not found'}")
    print(f"  Mobile     : {', '.join(p['mobiles']) or 'Not found'}")
    print(f"  Skills     : {', '.join(p['skills']) or 'None'}")
    print(f"  Experience : {p['experience']:g} year(s)")
    print(f"  Status     : {'ELIGIBLE' if is eligible(p) else 'NOT ELIGIBLE'}")
    print("-" * 60)

def main():
    print("=== STRUCTURED CANDIDATE PROFILES ===")
    profiles = [build_profile(r) for r in RESUMES]
    for i, p in enumerate(profiles, 1):
        show_profile(i, p)
    print("=== SHORTLISTED CANDIDATES (Experience >= 2 yrs and Python) ===")
    for p in filter(is_eligible, profiles):
        print(f"  {p['name']:<15} {p['experience']:g} yrs   {p['emails'][0]}")

if __name__ == "__main__":
    main()
```

Output Screenshot



Justification of the output

Each field has its own regular expression: an anchored multiline pattern for the name, a general e-mail pattern, a mobile pattern (optional +91, first digit 6–9), word-boundary skill matching (so “Java” does not match “JavaScript”) and an experience pattern that handles both “Experience: n years” and “n years of experience”. Priya (5 yrs, Python) and Ananya (2 yrs, Python) meet the criteria; Rahul has no Python skill and Karthik has only 1.5 years, so both are rejected.

Question 2 | Section 2: Regular Expressions – Product Search System

Question

An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

Answer *Python program: q2_product_search.py*

```
"""Q2 - Product Search System using Regular Expressions."""
import re

PRODUCTS = [
    "Apple iPhone 15", "Samsung Galaxy S24", "Sony Wireless Headphones",
    "boAt Bluetooth Speaker", "Dell Inspiron Laptop", "HP Pavilion Laptop",
    "Logitech Wireless Mouse", "Mechanical Keyboard", "Smart Watch Pro",
    "Smartphone Case", "Phone Stand", "Wireless Charger", "USB-C Cable",
    "Laptop Backpack", "Gaming Mouse Pad",
]

def build_pattern(keyword, mode):
    """Return the regex string for the requested search type."""
    k = re.escape(keyword)
    patterns = {
        "exact":   rf"\b{k}\b",       # keyword as a complete word
        "prefix":  rf"\b{k}\w*",      # word BEGINS with keyword
        "suffix":  rf"\w*{k}\b",      # word ENDS with keyword
        "partial": k,                 # keyword anywhere inside the name
    }
    return patterns[mode]

def search(products, keyword, mode, ignore_case=True):
    flags = re.IGNORECASE if ignore_case else 0
    regex = re.compile(build_pattern(keyword, mode), flags)
    return [p for p in products if regex.search(p)]

def run search(no, mode, keyword, ignore case, report):
    results = search(PRODUCTS, keyword, mode, ignore case)
    case = "Insensitive" if ignore case else "Sensitive"
    print(f"[{no}] {mode.upper():<8} '{keyword}'  case-{case.lower()}  -> {len(results)} match(es)")
    print(f"    regex   : {build pattern(keyword, mode)}")
    print(f"    results : {', '.join(results) if results else 'No match found'}")
    report.append((mode, keyword, case, len(results)))

def main():
    report = []
    print("======== PRODUCT SEARCH SYSTEM ========")
    tests = [("exact", "laptop", True), ("prefix", "smart", True),
             ("suffix", "phone", True), ("partial", "wire", True),
             ("exact", "MOUSE", True),     # case-insensitive search
             ("exact", "MOUSE", False),    # case-sensitive search -> no match
             ("prefix", "xyz", True)]
    for no, (mode, kw, ic) in enumerate(tests, 1):
        run search(no, mode, kw, ic, report)

    print("======== SEARCH SUMMARY REPORT ========")
    print(f"{'Search Type':<12}{'Keyword':<10}{'Case':<13}{'Matches':>7}")
    print("-" * 42)
    for mode, kw, case, n in report:
        print(f"{mode.title():<12}{kw:<10}{case:<13}{n:>7}")
    print("-" * 42)
    print(f"{'Total searches: ' + str(len(report)):<35}{sum(r[3] for r in report):>7}")
```

```
if __name__ == "__main__":
    main()
```

Output Screenshot

```
Terminal - q2_product_search.py

$ python q2_product_search.py
======== PRODUCT SEARCH SYSTEM ========
[1] EXACT    'laptop'  case-insensitive  -> 3 match(es)
    regex   : \blaptop\b
    results : Dell Inspiron Laptop, HP Pavilion Laptop, Laptop Backpack
[2] PREFIX   'smart'  case-insensitive  -> 2 match(es)
    regex   : \bsmart\w*
    results : Smart Watch Pro, Smartphone Case
[3] SUFFIX   'phone'  case-insensitive  -> 3 match(es)
    regex   : \w*phone\b
    results : Apple iPhone 15, Smartphone Case, Phone Stand
[4] PARTIAL  'wire'  case-insensitive  -> 3 match(es)
    regex   : wire
    results : Sony Wireless Headphones, Logitech Wireless Mouse, Wireless Charger
[5] EXACT    'MOUSE'  case-insensitive  -> 2 match(es)
    regex   : \bMOUSE\b
    results : Logitech Wireless Mouse, Gaming Mouse Pad
[6] EXACT    'MOUSE'  case-sensitive  -> 0 match(es)
    regex   : \bMOUSE\b
    results : No match found
[7] PREFIX   'xyz'  case-insensitive  -> 0 match(es)
    regex   : \bxyz\w*
    results : No match found
======== SEARCH SUMMARY REPORT ========
Search Type Keyword   Case         Matches
-----------------------------------------
Exact       laptop    Insensitive        3
Prefix      smart     Insensitive        2
Suffix      phone     Insensitive        3
Partial     wire      Insensitive        3
Exact       MOUSE     Insensitive        2
Exact       MOUSE     Sensitive          0
Prefix      xyz       Insensitive        0
-----------------------------------------
Total searches: 7                     13
```

Justification of the output

Exact search uses \bkeyword\b (whole word only), prefix uses \bkeyword\w*, suffix uses \w*keyword\b and partial search matches the keyword anywhere. re.IGNORECASE makes the search case-insensitive: “MOUSE” finds 2 products, while the same search with case-sensitivity returns 0. The summary report lists the number of matches for every search (13 in total).

Question 3 | Section 3: Regular Expressions – University Registration System

Question

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Answer *Python program: q3_university_registration.py*

```
"""Q3 - University Registration Validation System using Regular Expressions."""
import re

RULES = {
    "Register Number": (r"^19\d{7}$",
                        "9 digits starting with 19"),
    "Email":           (r"^[A-Za-z][A-Za-z0-9._-]*@saveetha\.com$",
                        "use name@saveetha.com"),
    "Course Code":     (r"^[A-Z]{3}\d{2,4}$",
                        "e.g. DSA03 (3 capitals + 2-4 digits)"),
    "Semester":        (r"^[1-8]$",
                        "must be 1 to 8"),
    "Mobile Number":   (r"^[6-9]\d{9}$",
                        "10 digits starting with 6-9"),
}

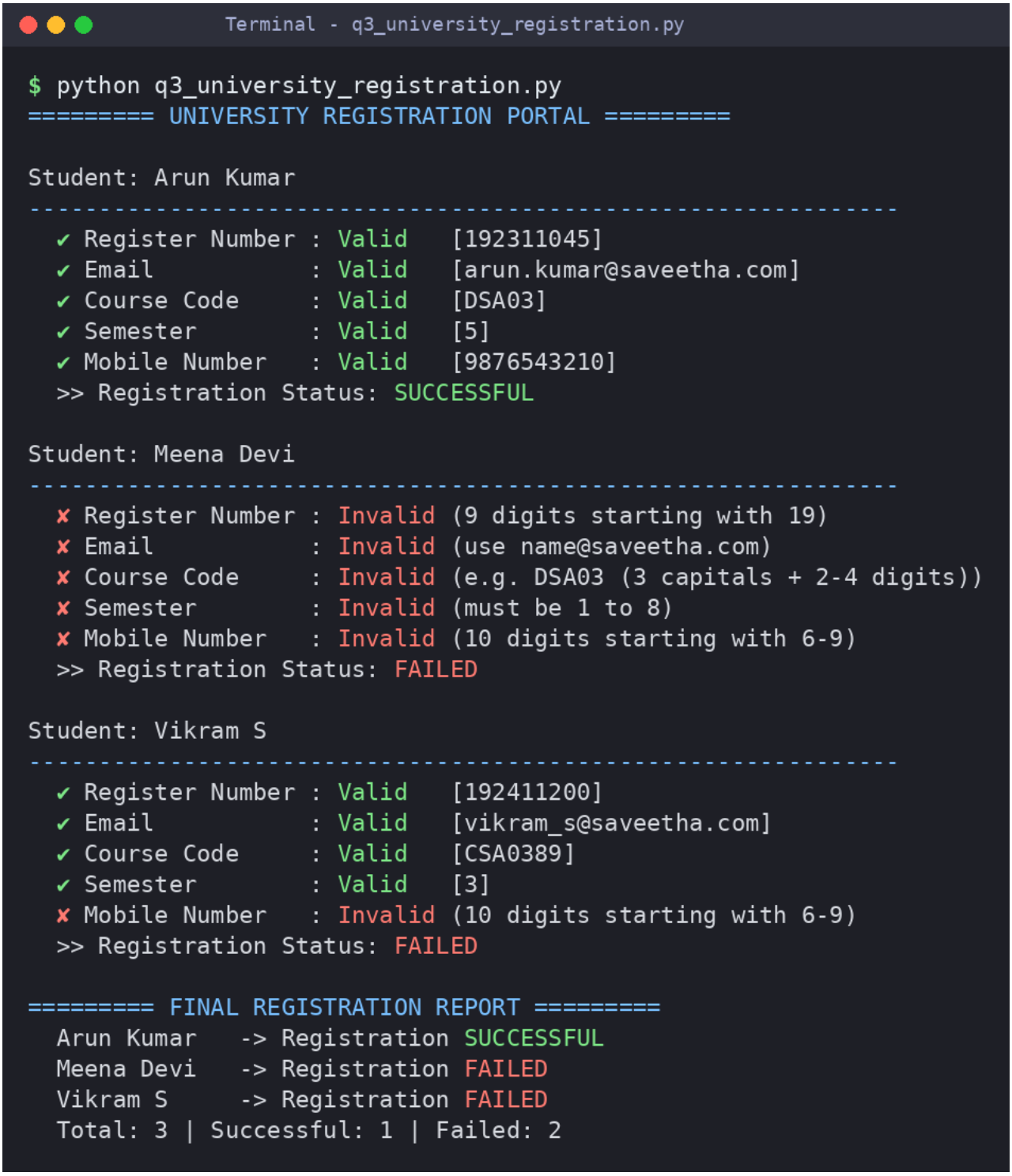
def validate_field(field, value):
    pattern, hint = RULES[field]
    if re.fullmatch(pattern, value):
        return True, f"✓ {field:<16}: Valid"
    return False, f"✗ {field:<16}: Invalid ({hint})"

def register_student(student):
    print(f"\nStudent: {student['Name']}")
    print("-" * 62)
    all valid = True
    for field in RULES:
        ok, message = validate field(field, student[field])
        print(f"  {message}   [{student[field]}]" if ok else f"  {message}")
        all valid &= ok
    status = "SUCCESSFUL" if all valid else "FAILED"
    print(f"  >> Registration Status: {status}")
    return all valid

def main():
    students = [
        {"Name": "Arun Kumar", "Register Number": "192311045", "Email": "arun.kumar@saveetha.com",
         "Course Code": "DSA03", "Semester": "5", "Mobile Number": "9876543210"},
        {"Name": "Meena Devi", "Register Number": "19231", "Email": "meena@gmail.com",
         "Course Code": "dsa3", "Semester": "9", "Mobile Number": "5123456789"},
        {"Name": "Vikram S", "Register Number": "192411200", "Email": "vikram s@saveetha.com",
         "Course Code": "CSA0389", "Semester": "3", "Mobile Number": "98765"},
    ]
    print("========= UNIVERSITY REGISTRATION PORTAL =========")
    results = [register student(s) for s in students]
    print("\n========= FINAL REGISTRATION REPORT =========")
    for s, ok in zip(students, results):
        print(f"  {s['Name']:<12} -> {'Registration SUCCESSFUL' if ok else 'Registration FAILED'}")
    ok = sum(results)
    print(f"  Total: {len(results)} | Successful: {ok} | Failed: {len(results) - ok}")

if   name   == "  main  ":
    main()
```

Output Screenshot



Justification of the output

re.fullmatch is used so the whole value must obey its pattern. Arun's details pass all five checks (registration successful); Meena fails all five; Vikram fails only the mobile number (5 digits). A registration is successful only when every field is valid, and each failed field shows a message explaining the expected format.